Generación de Servicios en Red

Unidad de Trabajo 4

Contenidos

- Identificar protocolos estándar de comunicación en red.
- Analizar librerías Java para la implementación de servicios en red.
- Desarrollar servicios en red funcionales.
- Incorporar comunicación simultánea con múltiples clientes.
- Probar los servicios con clientes reales.
- Verificar la disponibilidad del servicio.
- Mejorar eficiencia y robustez.
- Documentar y depurar el servicio desarrollado.

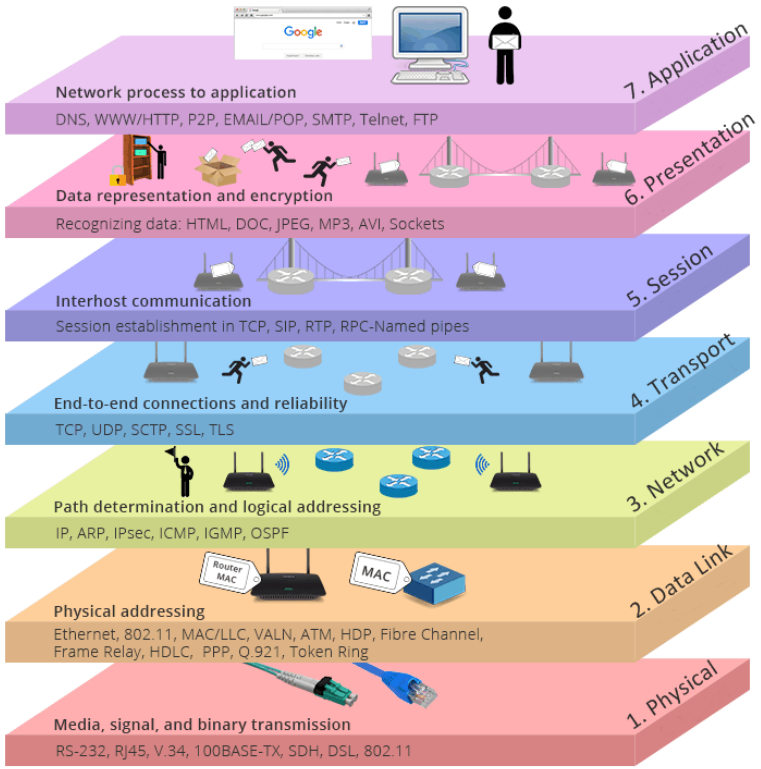
¿Qué es un servicio?

- Es el conjunto de mecanismos que hacen posible la función de un sistema.
- Los usuarios interactúan con el sistema y hacen uso de sus servicios a través de la interfaz del servicio.

Servicios a nivel de aplicación

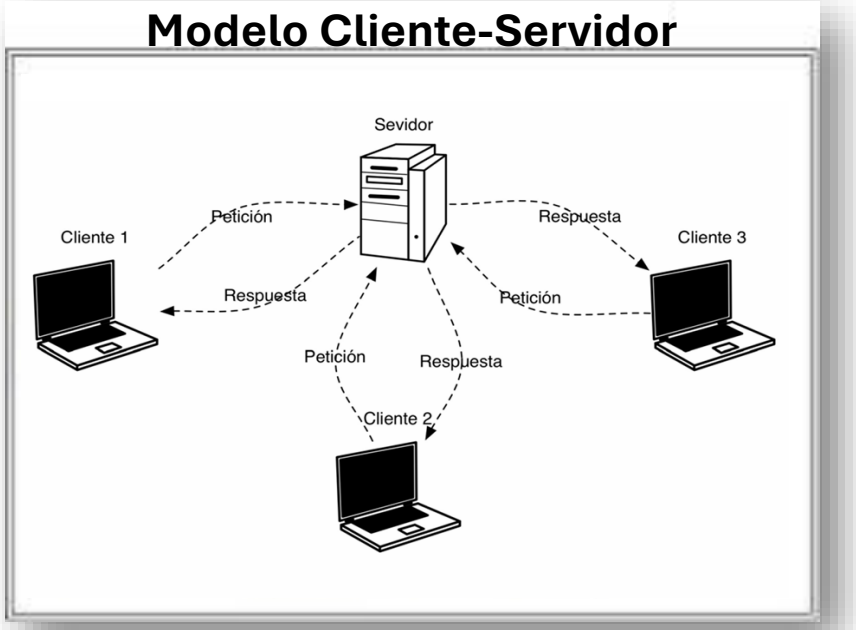
- El nivel más alto de la pila IP, lo componen las aplicaciones que conforman un sistema distribuido.

Un **protocolo de nivel de aplicación** es el conjunto de reglas que gobiernan la interacción entre los diferentes elementos de una aplicación distribuida..



Modelo Cliente / Servidor

- A la hora de programar una aplicación siguiendo el modelo cliente/servidor, se deben definir de forma precisa los siguientes aspectos:
  - Funciones del servidor.
  - Tecnología de comunicaciones.
  - Protocolo de nivel de aplicación.



Funciones del servidor

- Definir de forma clara qué hace el servidor, debemos partir de cuestiones como:
  - ¿Cuál es la **función básica** de nuestro servidor?
  - El servicio que proporciona nuestro servidor, ¿es **rápido o lento**?
  - El servicio que proporciona nuestro servidor, ¿puede resolverse con una simple **petición y respuesta o** requiere del intercambio de **múltiples mensajes** entre este y el cliente?
  - ¿Debe ser capaz nuestro servidor de atender a **varios clientes simultáneamente**?

Tecnología de comunicaciones

- Se debe escoger la tecnología de comunicaciones que es necesario utilizar. Los dos mecanismos básicos de comunicación estudiados son **sockets stream y datagram**.
- Cada tipo de socket presenta una serie de **ventajas e inconvenientes**.
- Es necesario escoger el tipo de socket adecuado, **según las características del servicio** que proporciona nuestro servidor.
  - Sockets stream: más fiables y orientados a conexión, por lo que se deben usar en aplicaciones complejas en las que clientes y servidores intercambian muchos mensajes.
  - Sockets datagram: menos fiables, pero más eficientes, por lo que es preferible usarlos cuando las aplicaciones son sencillas, y no es problema que se pierda algún mensaje.

Protocolo de nivel de aplicación

- **Conjunto de reglas** que gobiernan la interacción entre los diferentes elementos de una aplicación distribuida.
- Se debe definir explícitamente el **formato de los mensajes** que se intercambian entre cliente y servidor, **y las posibles secuencias** en las que estos pueden ser enviados y recibidos.
- La definición del protocolo de nivel de aplicación **debe contener todos los posibles tipos de mensajes** (tanto peticiones como respuestas) que pueden ser enviados y recibidos, **indicando cuándo se puede enviar** cada uno y cuándo no.

Protocolo de nivel de aplicación

- Protocolos sin estado (**stateless**): la secuencia concreta en la que se reciben los mensajes no es importante, ya que no afecta al resultado de estos. El servidor se limita a recibir las peticiones del cliente y a responderlas una por una, **de forma independiente**.
- Protocolos con estado (**stateful**): la secuencia concreta en la que se reciben los mensajes es importante, ya que afecta al resultado final de las peticiones. **El servidor debe almacenar información intermedia durante la sesión (estado de la sesión)**. Conocer este estado es imprescindible para poder resolver las peticiones de manera correcta.

Protocolo de nivel de aplicación

- En muchos casos **el cliente y el servidor de sistemas distribuidos modernos son aplicaciones independientes**, desarrolladas por diferentes personas/compañías y muchas veces ni siquiera implementadas usando el mismo lenguaje de programación.
- La definición exhaustiva de **un protocolo de nivel de aplicación es fundamental** para garantizar que clientes y servidores pueden comunicarse de manera efectiva.
- Para la mayoría de las aplicaciones distribuidas comunes se han definido, a lo largo de los años, **protocolos de nivel de aplicación estándar**, que facilitan la tarea de desarrollar servidores y clientes para ellas.

Clasificación general de protocolos del nivel de aplicación

- **Transferencia de datos:** FTP, FTPS, SFTP
- **Servicios web / APIs:** HTTP, HTTPS, REST, SOAP
- **Comunicación persistente:** WebSocket
- **Mensajería:** SMTP, POP3, IMAP
- **Servicios de red:** DNS, DHCP
- **Acceso remoto:** SSH, Telnet
- **Comunicación ligera:** MQTT
- **Transporte en tiempo real:** RTP / RTSP

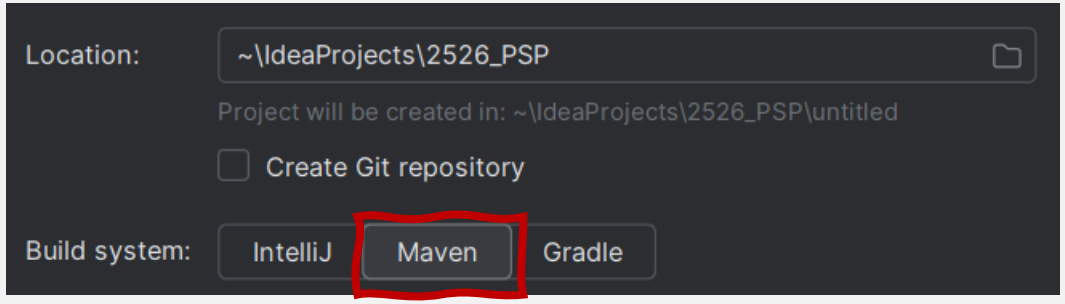
Protocolo	Descripción breve	Uso típico	Ventajas	Inconvenientes	Uso actual
HTTP	Protocolo cliente-servidor sin estado para intercambio de recursos	APIs REST, servicios web, microservicios	Universal, fácil de usar, gran soporte de librerías	No persistente, sobrecarga de cabeceras	**Muy alto**
HTTPS	HTTP cifrado mediante TLS	APIs seguras, comercio electrónico	Seguridad, confidencialidad, autenticación	Ligera sobrecarga por cifrado	**Muy alto**
FTP	Transferencia de archivos en texto plano	Subida/descarga de ficheros	Simple, ampliamente soportado	Inseguro, problemas con firewalls	**Bajo**
FTPS	FTP con cifrado SSL/TLS	Transferencia segura de archivos	Cifrado, compatibilidad con FTP	Configuración compleja, puertos múltiples	**Medio**
SFTP	Transferencia de archivos sobre SSH	Transferencia segura, automatización	Seguro, usa un solo puerto	No compatible con FTP clásico	**Alto**
WebSocket	Comunicación bidireccional persistente	Chats, juegos, tiempo real	Baja latencia, tiempo real	Mayor consumo de recursos	**Alto**
SMTP	Envío de correo electrónico	Servidores de correo, notificaciones	Estándar global, robusto	No sirve para recibir correo	**Medio**
POP3	Descarga de correos al cliente	Clientes de correo simples	Sencillo, bajo consumo	No sincroniza múltiples dispositivos	**Medio-bajo**
IMAP	Acceso remoto y sincronizado al correo	Correo web, móviles	Sincronización, carpetas en servidor	Más consumo de recursos	**Alto**
MQTT	Mensajería ligera publish/subscribe	IoT, sensores, sistemas distribuidos	Muy ligero, eficiente	Requiere broker, no web	**Alto**
SSH	Acceso remoto seguro	Administración de sistemas	Cifrado, autenticación fuerte	No orientado a servicios genéricos	**Muy alto**
Telnet	Acceso remoto sin cifrado	Sistemas legacy, pruebas educativas	Muy simple	Inseguro, obsoleto	**Muy bajo**
DNS	Resolución de nombres a direcciones IP	Internet, redes locales	Transparente, esencial	Vulnerable sin DNSSEC	**Muy alto**
DHCP	Asignación dinámica de IPs	Redes corporativas y domésticas	Automatiza configuración	Dependencia del servidor	**Muy alto**

Librería Java

Protocolo	Librerías Java estándar (JDK)	Librerías Java externas recomendadas
HTTP	java.net.http.HttpClient, HttpServer	Spring Boot (REST), Apache HttpClient, Retrofit
HTTPS	java.net.http.HttpClient (TLS)	Spring Boot, OkHttp
FTP	—	Apache Commons Net (FTPClient)
FTPS	—	Apache Commons Net (FTPSClient)
SFTP	—	JSch, Apache Mina SSHD
WebSocket	java.net.http.WebSocket	Spring WebSocket, Tyrus
SMTP	—	Jakarta Mail (JavaMail)
POP3	—	Jakarta Mail
IMAP	—	Jakarta Mail
MQTT	—	Eclipse Paho MQTT
SSH	—	JSch, Apache Mina SSHD
Telnet	—	Apache Commons Net (TelnetClient)
DNS	InetAddress	dnsjava
DHCP	—	Apache Commons Net (DHCP)

Desarrollo de servicio SFTP

- Creamos un nuevo proyecto seleccionando Maven como sistema de gestión de librerías.



- Esto nos permitirá integrar fácilmente en el proyecto las librerías necesarias desde el repositorio de Maven.

Desarrollo de servicio SFTP

- Añadimos las dependencias Maven en el fichero de configuración

pom.xml

```xml
<dependencies>
    <!-- Servidor SFTP -->
    <dependency>
      <groupId>org.apache.sshd</groupId>
      <artifactId>sshd-sftp</artifactId>
      <version>2.11.0</version>
    </dependency>

    <!-- Cliente SFTP -->
    <dependency>
      <groupId>com.jcraft</groupId>
      <artifactId>jsch</artifactId>
      <version>0.1.55</version>
    </dependency>
</dependencies>
```

- Para el servidor FTP (Apache MINA SSHD), la librería:

  org.apache.sshd → sshd.sftp

- Para el cliente FTP (JSCH), la librería:

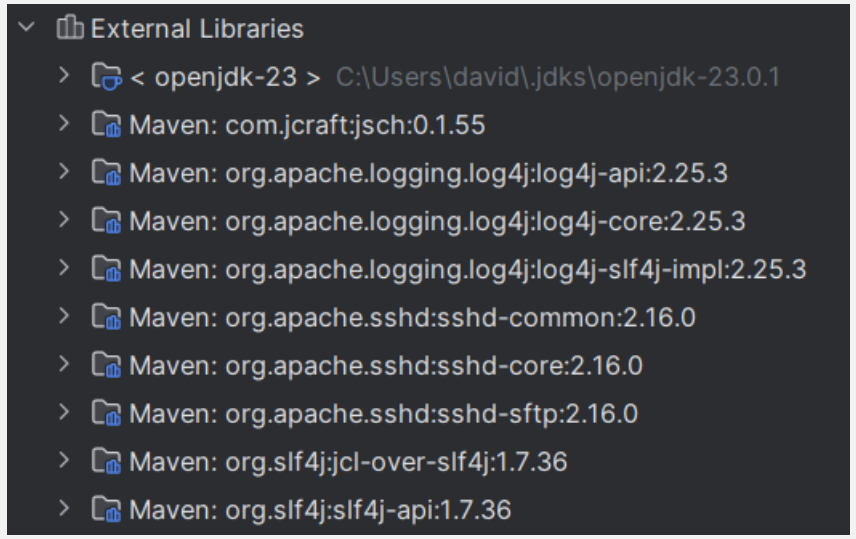
  com.jcraft → jsch

Desarrollo de servicio SFTP

- Al cambiar pom.xml se debe actualizar el proyecto Maven pulsando

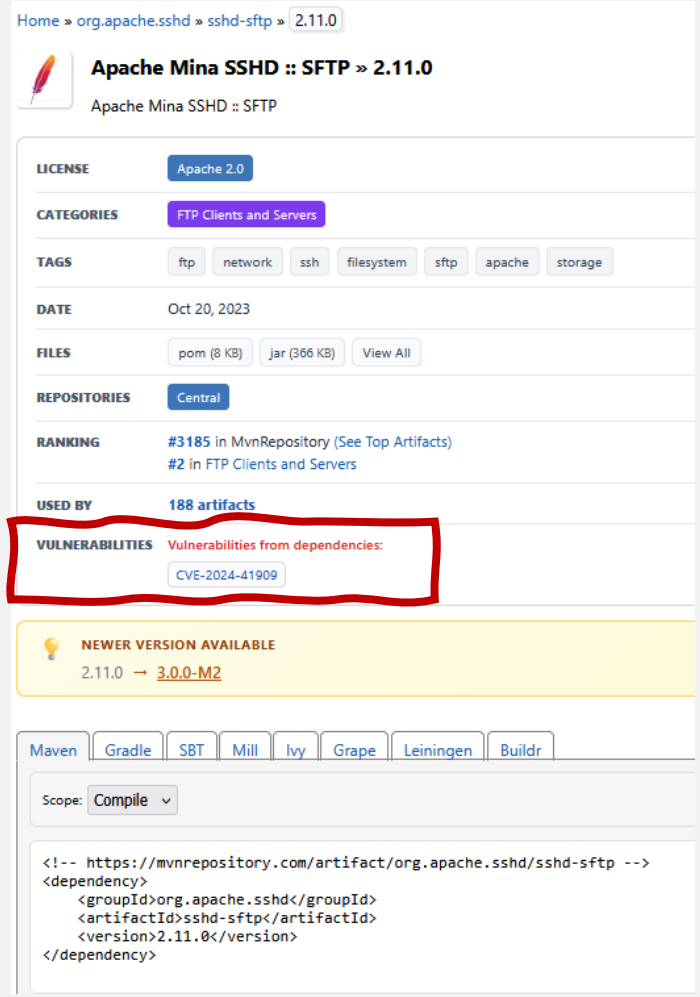
```
<project xmlns="http://maven.apache.org/POM/4.0.0"
    <properties>
    </properties>

    <dependencies>
        <!-- Servidor SFTP -->
        <dependency>
            <groupId>org.apache.sshd</groupId>
            <artifactId>sshd-sftp</artifactId>
            <version>2.16.0</version>
        </dependency>
```

- El listado de librerías externas se actualiza



Desarrollo de servicio SFTP

- La versión elegida de la librería es MUY importante.
  - Compatibilidad
  - Actualización de funcionalidades
  - Vulnerabilidades de seguridad *(usad la versión 2.16.0 de la librería sshd-sftp)*



Desarrollo de servicio SFTP

- Para evitar el warning sobre registro de logs con SLF4J:

pom.xml

```
<dependencies>

...
    <!– Registro de logs -->
    <dependency>
        <groupId>org.apache.logging.log4j</groupId>
        <artifactId>log4j-slf4j-impl</artifactId>
        <version>2.25.3</version>
        <scope>compile</scope>
    </dependency>

...
</dependencies>
```

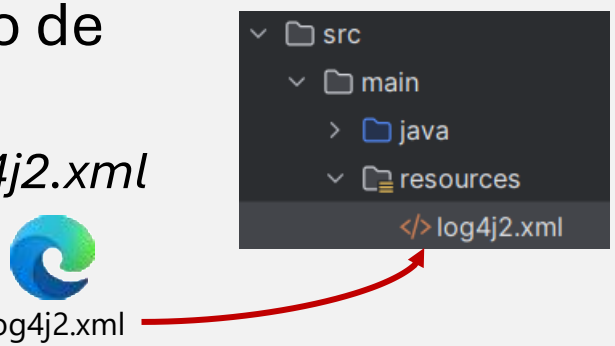
- Log4J permite el registro de los logs

  org.apache.logging.log4j → *log4j-slf4j-impl*

```
// Registramos logs usando Log4j
Logger log = LoggerFactory.getLogger(ServidorSFTP.class);
log.info("Servidor SFTP iniciado en puerto 2222");
```

- Se requiere el fichero de configuración

  src/main/resources/log4j2.xml



Desarrollo de servicio SFTP

ServidorSFTP.java

```
SshServer sshd = SshServer.setUpDefaultServer();
sshd.setPort(2222);

// Clave del servidor.
sshd.setKeyPairProvider(
                new SimpleGeneratorHostKeyProvider(Path.of("hostkey.ser"))
);

// Autenticación explícita con PasswordAuthenticator
sshd.setPasswordAuthenticator(new PasswordAuthenticator() {
   @Override
   public boolean authenticate(
         String username,
         String password,
         ServerSession session) {
      return username.equals("usuario")
            && password.equals("password");
   }
});
```

- Qué hace este servidor:
  - Escucha en el puerto 2222
  - Se identifica con clave púb/priv.
  - Autenticación usuario/contraseña
  - Expone solo el directorio data/
  - Permite múltiples clientes simultáneos.

```
// Subsystem SFTP
sshd.setSubsystemFactories(
      Collections.singletonList(new SftpSubsystemFactory())
);
// Directorio raíz del servidor. Se fija la carpeta "data"
VirtualFileSystemFactory vfs = new VirtualFileSystemFactory();
vfs.setDefaultHomeDir(Path.of("data"));
sshd.setFileSystemFactory(vfs);
// Arranca el servidor SSH
sshd.start();
```

Desarrollo de servicio SFTP

ClienteSFTP.java

```
JSch jsch = new JSch();
session = jsch.getSession(user, host, port);
session.setPassword(password);

// Desactivar comprobación de host (solo para pruebas)
session.setConfig("StrictHostKeyChecking", "no");

session.connect();

ChannelSftp sftp = (ChannelSftp) session.openChannel("sftp");
sftp.connect();

// Subir fichero
sftp.put(
    new FileInputStream(srcDir + "/ejemplo.txt"),
    "ejemplo_subido.txt"
);

sftp.disconnect();
session.disconnect();
```

- Qué hace este cliente:
  - Se conecta al servidor SFTP
  - Autentica usuario
  - Sube un archivo al servidor
  - Cierra recursos correctamente.

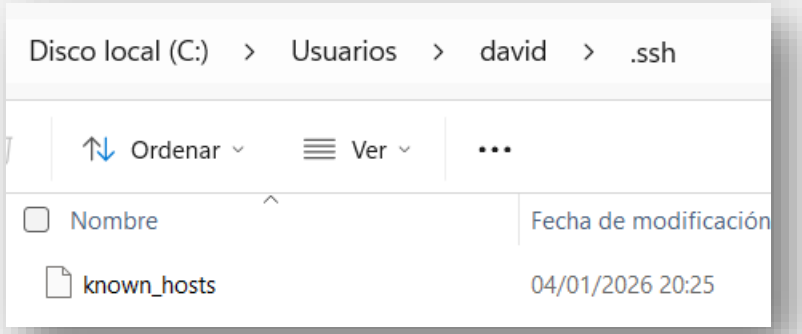
```
// Subsystem SFTP
sshd.setSubsystemFactories(
    Collections.singletonList(new SftpSubsystemFactory())
);

// Directorio raíz del servidor
sshd.setFileSystemFactory(session -> Path.of("data"));

sshd.start();
```

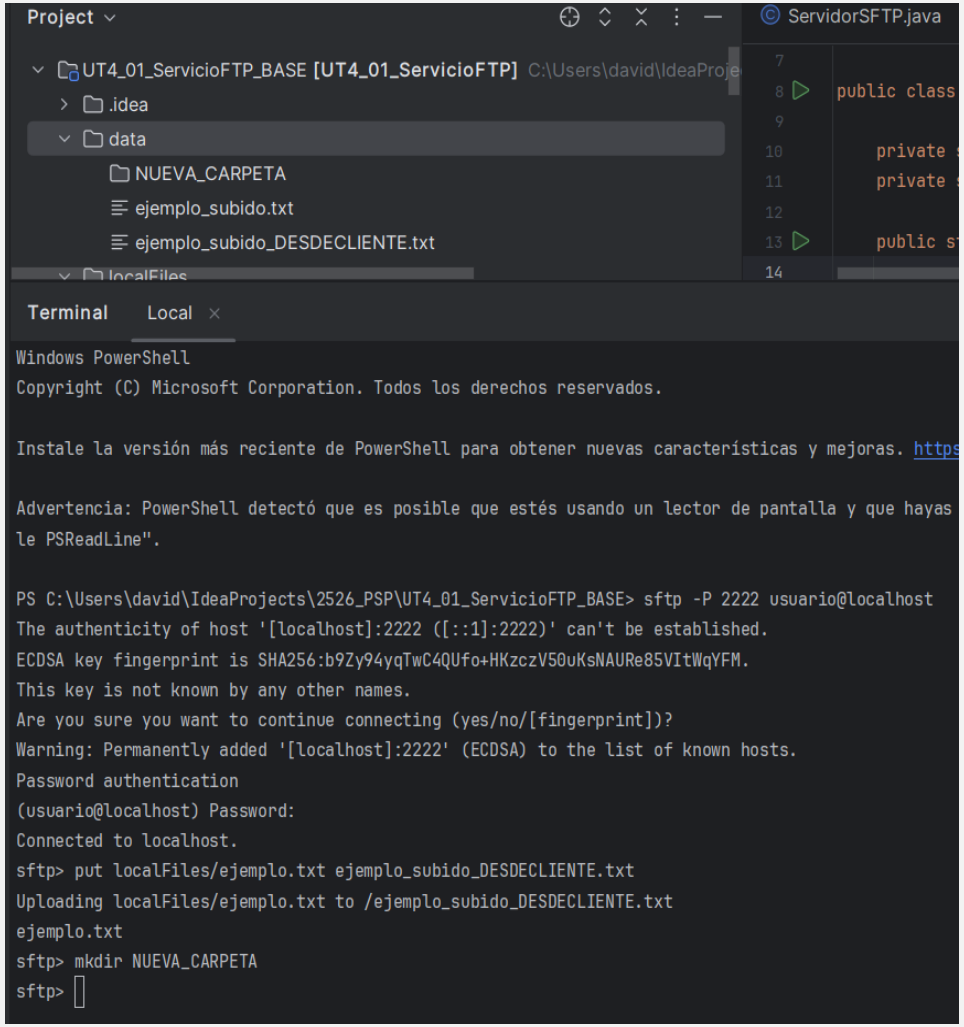
Desarrollo de servicio SFTP

- Podéis conectar por consola al servidor usando el cliente SFTP del sistema:

  sftp -P 2222 usuario@localhost

- La clave se almacena en la carpeta .ssh del usuario, en el fichero known_hosts.





Desarrollo de servicio SFTP

- **Persistencia de la identidad del servidor:**
  - El servidor mantiene la misma clave entre reinicios.
  - Clientes que ya han confiado en el servidor recibirán advertencias de cambio de clave.
- **Seguridad:**
  - Si alguien accede a hostkey.ser, podría suplantar el servidor.
  - Por eso **el archivo debe protegerse**, usualmente con permisos de solo lectura para el usuario del proceso SSHD.
- **Simplicidad vs. control:**
  - SimpleGeneratorHostKeyProvider es muy conveniente para desarrollo o pruebas.
  - En producción, normalmente se usan **claves preexistentes en formato OpenSSH** para mayor control y compatibilidad.
- **Impacto en la conexión con el cliente:**
  - Cuando un cliente se conecta, el servidor usa la clave de hostkey.ser para identificarse.
  - El cliente puede verificar la clave y establecer cifrado seguro.

Técnicas avanzadas de programación distribuida

- Los *stream/datagram* ***sockets*** **son la base de la mayoría de los mecanismos de comunicación** entre aplicaciones.
- No obstante, las interfaces de programación que normalmente se usan para operar con ellos resultan poco practicas, ya que **requieren el formateo completo de los mensajes y el control detallado de las comunicaciones**.
- Para programar aplicaciones mas sofisticadas se hace uso de **técnicas más avanzadas**. Internamente hacen uso de *sockets*, pero **los abstraen utilizando mecanismos de mas alto nivel**, que incorporan funcionalidades adicionales.

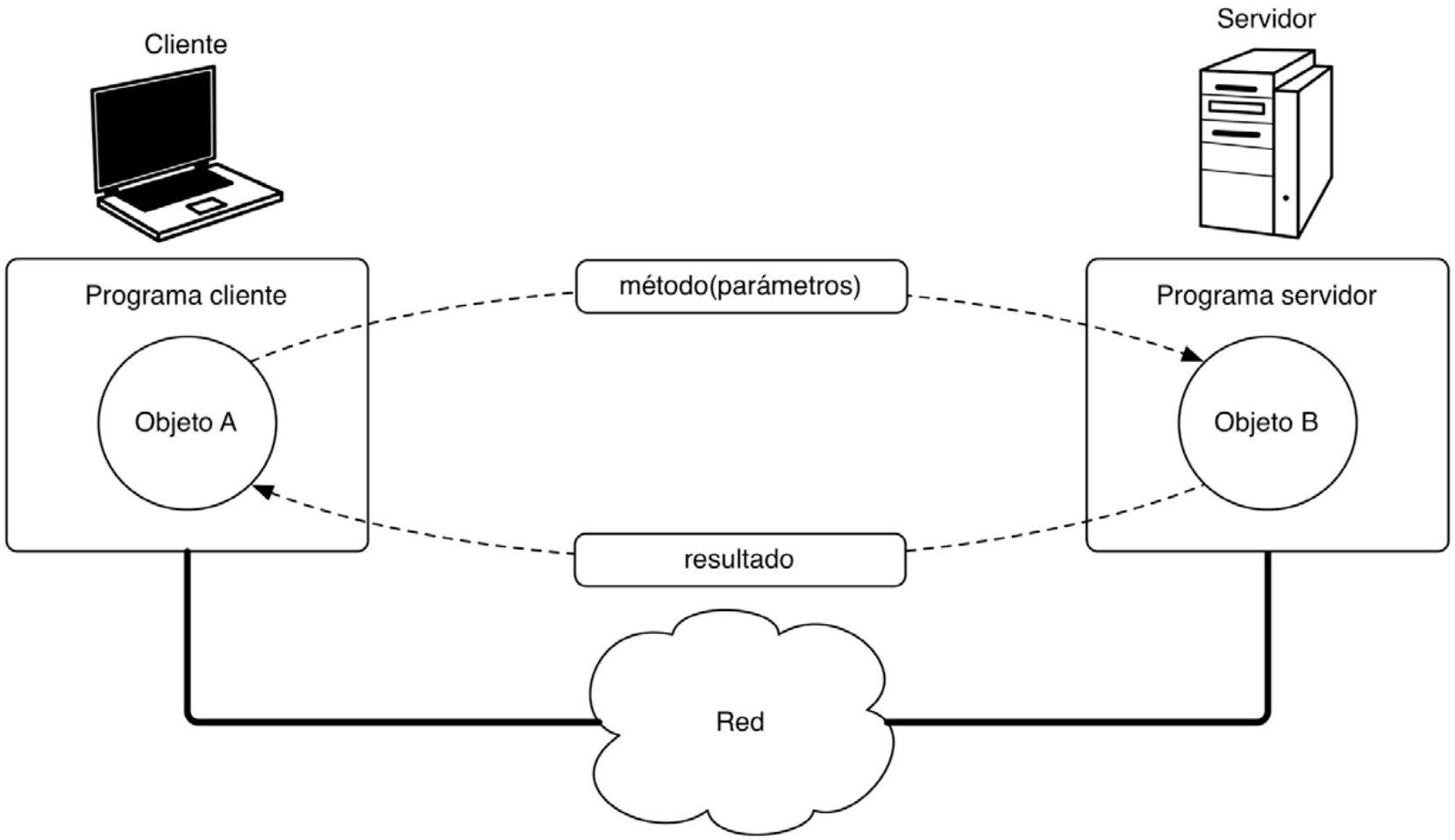
Invocación de métodos remotos (RMI)

En un programa orientado a objetos la invocación de un método se puede ver como un proceso de comunicación:

- El objeto A envía un mensaje al B a modo de petición (invocación del método).
- El objeto B procesa la petición (ejecución del método) contesta con un mensaje de respuesta (valor de retorno del método).

La **invocación de métodos remotos** funciona de la misma forma, pero cada objeto (A y B) se encuentran en máquinas distintas.

Invocación de métodos remotos (RMI)



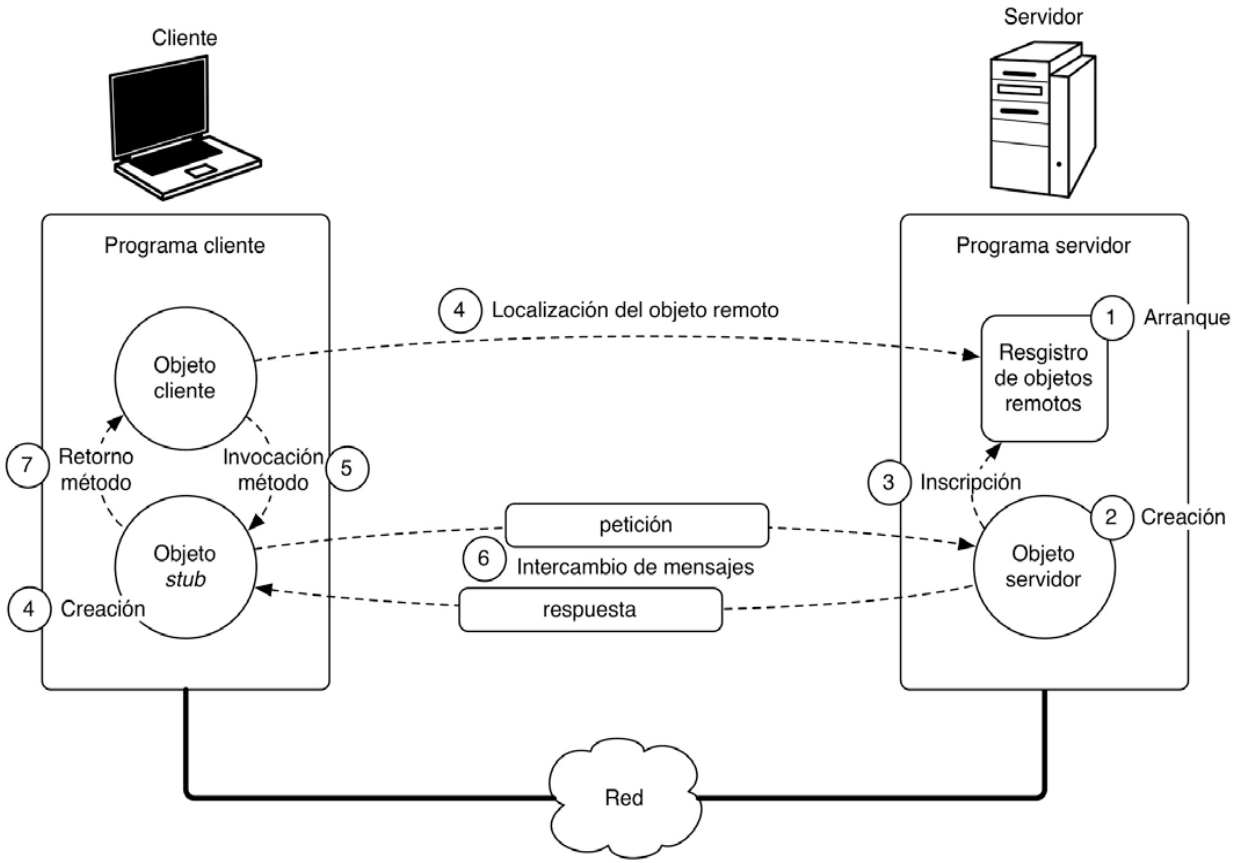
- Objeto servidor: El objeto cuyo método es invocado. También se le llama **objeto remoto**.
- Objeto cliente: El objeto que realiza la petición.
- Método invocado: El mensaje de petición, incluyendo los parámetros del método.
- Valor de retorno: El mensaje de respuesta.

Invocación de métodos remotos (RMI)

En Java, para llevar a cabo invocaciones de métodos remotos, debe:

- Existir una **infraestructura que capture las llamadas y genere el tráfico en la red**.
- Esta infraestructura tendrá dos componentes:
  - **Objetos stub**: Sustituyen a los objetos remotos en el programa cliente. Convierten las invocaciones de métodos en mensajes y los transfieren por la red.
  - **Registro de objetos remotos**: Es un servicio de nivel de aplicación que sirve para poder localizar objetos remotos.

Invocación de métodos remotos (RMI)



1. (servidor) Arranque del registro de objetos remotos
2. (servidor) Creación del objeto servidor.
3. (servidor) Inscripción del objeto servidor en el registro.
4. (cliente) Localización del objeto remoto y creación del objeto *stub*.
5. (cliente) Invocación del método del objeto *stub*.
6. Intercambio de mensajes entre servidor y cliente.
7. (cliente) Obtención del valor de retorno del método.

Invocación de métodos remotos (RMI)

```
public interface RMICalcInterface extends Remote {
    public int suma(int a, int b) throws RemoteException;
    public int resta(int a, int b) throws RemoteException;
    public int multip(int a, int b) throws RemoteException;
    public int div(int a, int b) throws RemoteException;
}
```

```
public class RMICalcServer implements RMICalcInterface
```

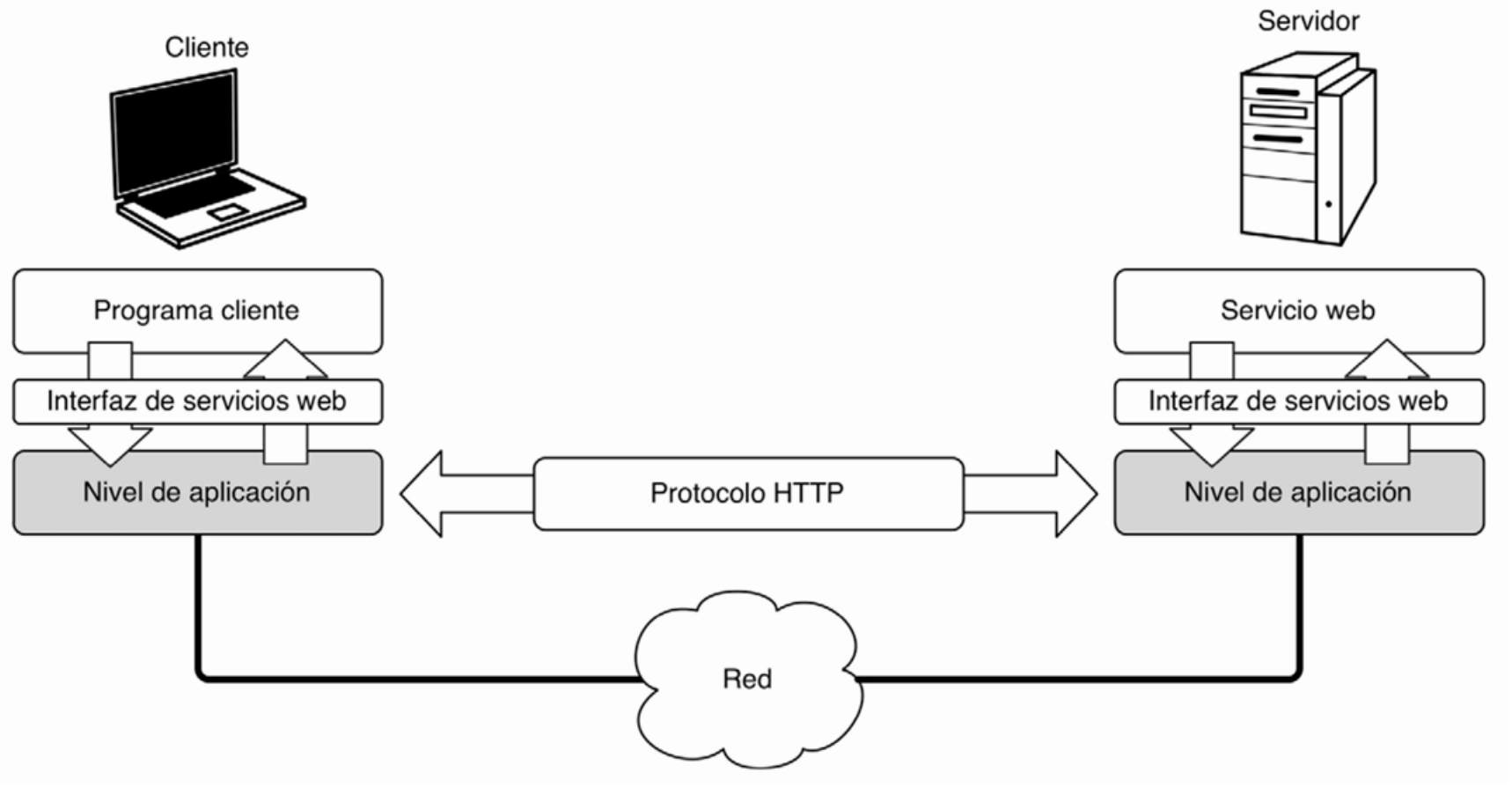
```
public class RMICalcClient {
    public static void main(String[] args) {
        RMICalcInterface calc = null;
        try {
            Registry registry = LocateRegistry.getRegistry("localhost", 5555);
            calc = (RMICalcInterface) registry.lookup("Calculadora");
        } catch (Exception e) {
            e.printStackTrace();
        }
        if (calc != null) {
            try {
                System.out.println("2 + 2 = " + calc.suma(2, 2));
                System.out.println("99 - 45 = " + calc.resta(99, 45));
                System.out.println("125 * 3 = " + calc.multip(125, 3));
                System.out.println("1250 / 5 = " + calc.div(1250, 5));
            } catch (RemoteException e) {
                e.printStackTrace();
```

Servicios Web REST / SOAP

Un **servicio web** es una aplicación distribuida, basada en el modelo de comunicaciones cliente/servidor, que utiliza el protocolo de nivel de aplicación HTTP para la transferencia de mensajes. Son especialmente relevantes:

- Servicios Web SOAP:
  - Representan los mensajes usando el leguaje SOAP (*Simple Object Access Protocol*), basado en XML.
  - Facilidad de desarrollo e interoperabilidad entre diferentes clientes y servidores.
- Servicios Web REST:
  - Representan los mensajes con REST (Representational State Transfer).
  - Menor interoperabilidad que con SOAP, pero mejor rendimiento.

Servicios Web REST / SOAP



El uso del protocolo HTTP/HTTPS facilita la interoperabilidad entre clientes y servidores.

Servicios
Web
REST / SOAP

Servicios Web REST:

Un **servicio web REST** es una **arquitectura** que:

- Usa **HTTP como protocolo.**
- **Expone recursos** mediante URLs.
- Usa comandos HTTP:
  - GET
  - POST
  - PUT
  - DELETE
- Devuelve representaciones de los recursos (p.e.: en formato JSON).

Servicios Web REST / SOAP

Servicios Web SOAP:

Un servicio web SOAP es un **protocolo estándar** que :

- Usa **XML** como formato de intercambio de datos.
- Define estrictamente la estructura de los mensajes.
- Utiliza un **contrato WSDL** para describir las operaciones disponibles.
- Puede funcionar sobre distintos protocolos de transporte:
  - HTTP / HTTPS
  - SMTP
  - JMS
- Implementa mecanismos avanzados de:
  - Seguridad (WS-Security)
  - Transacciones
  - Fiabilidad